

LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK



Disusun Oleh:

Gina Ramadhani

NIM: 2511533014

Kelas: B

Dosen Pengampu: Nurfiah, S.ST, M.Kom

Asisten Praktikum: Siti Fadhillah Rahmi

DAPERTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum dengan judul “Class, Object, Constructor dan Method: Interaksi Objek & Manipulasi State (Mini Bank)” ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi kegiatan praktikum pemrograman berorientasi objek.

Praktikum ini membahas konsep dasar pemrograman berorientasi objek, khususnya class, object, constructor, dan method melalui simulasi sederhana sistem perbankan. Dengan praktikum ini, diharapkan mahasiswa dapat memahami bagaimana objek saling berinteraksi serta bagaimana state suatu objek dapat dimanipulasi melalui method. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran sangat diharapkan untuk penyempurnaan laporan ini.

Padang, September 2026

Penulis

DAFTAR PUSTAKA

KATA PENGANTAR.....	i
DAFTAR PUSTAKA.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN.....	3
A. Langkah Kerja Praktikum	3
B. Tugas/Latihan Praktikum	11
BAB III PENUTUP.....	16
3.1 Kesimpulan.....	16
DAFTAR PUSTAKA.....	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman berorientasi objek merupakan salah satu konsep penting dalam pengembangan perangkat lunak yang memungkinkan program disusun berdasarkan objek dan perilaku yang dimilikinya. Konsep seperti class, object, constructor, dan method menjadi dasar dalam membangun program yang terstruktur dan mudah dikembangkan. Pemahaman terhadap konsep tersebut diperlukan agar mahasiswa mampu membuat program yang merepresentasikan permasalahan di dunia nyata ke dalam bentuk objek.

Salah satu contoh penerapannya adalah simulasi sistem perbankan sederhana atau Mini Bank. Dalam program tersebut, objek dapat digunakan untuk merepresentasikan nasabah atau rekening, sedangkan method digunakan untuk melakukan berbagai operasi seperti penyetoran, penarikan, dan pengecekan saldo. Melalui praktikum ini, mahasiswa dapat memahami interaksi antarobjek sekaligus mempelajari bagaimana state atau keadaan suatu objek dapat berubah berdasarkan operasi yang dilakukan.

1.2 Tujuan Praktikum

1. Memahami konsep dasar class, object, constructor, dan method dalam pemrograman berorientasi objek.
2. Mampu membuat dan menggunakan object berdasarkan class yang telah dibuat.
3. Memahami cara melakukan manipulasi state pada suatu object melalui method.
4. Menerapkan konsep interaksi objek dalam pembuatan simulasi Mini Bank sederhana.

1.3 Manfaat Praktikum

1. Meningkatkan pemahaman mengenai konsep dasar pemrograman berorientasi objek.
2. Melatih kemampuan membuat class, object, constructor, dan method dalam program Java.

3. Membantu memahami perubahan state suatu object berdasarkan proses yang dilakukan.
4. Memberikan pengalaman dalam menerapkan konsep OOP pada permasalahan sederhana yang menyerupai sistem perbankan.

BAB II

PEMBAHASAN

A. Langkah Kerja Praktikum

a) Class Rekening

1. Buatlah Sebuah Folder dengan nama PrakPBO_Kelas_NIM
2. Kemudian buat package, pada praktikum ini saya menamainya dengan pekan1
3. Setelah itu buat class didalam package tersebut dengan nama Rekening

```
1 package pekan1;
2 import java.util.Scanner;
3 public class Rekening {
```

4. Tambahkan State/Atribut

```
4     String nomorRekening;
5     String namaPemilik;
6     double saldo;
```

Baris 4 → public class Rekening { digunakan untuk membuat class bernama Rekening sebagai cetakan untuk membuat objek rekening.

Baris 5 → String nomorRekening; digunakan untuk membuat variabel nomorRekening bertipe String untuk menyimpan nomor rekening.

Baris 6 → String namaPemilik; digunakan untuk membuat variabel namaPemilik bertipe String untuk menyimpan nama pemilik rekening.

5. Buat constructor

```
8 public Rekening(String nomor, String nama, double saldoAwal) {
9     nomorRekening = nomor;
10    namaPemilik = nama;
11    saldo = saldoAwal;
12    System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
13 }
```

Baris 8 → public Rekening(String nomor, String nama, double saldoAwal) { digunakan untuk membuat constructor Rekening yang menerima nomor rekening, nama pemilik, dan saldo awal.

Baris 9 → nomorRekening = nomor; digunakan untuk memasukkan nilai parameter nomor ke dalam variabel nomorRekening.

Baris 10 → namaPemilik = nama; digunakan untuk memasukkan nilai parameter nama ke dalam variabel namaPemilik.

Baris 11 → saldo = saldoAwal; digunakan untuk memasukkan nilai saldo awal ke dalam variabel saldo.

Baris 12 → `System.out.println(...)` digunakan untuk menampilkan pesan bahwa rekening berhasil dibuat beserta nama pemilik dan saldo awal.

6. Buat method/behavior

```
14 public void setorTunai(double nominal) {  
15     if (nominal > 0) {  
16         saldo += nominal;  
17         System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);  
18     } else {  
19         System.out.println("Gagal: Nominal setor harus lebih dari 0!");  
20     }  
21 }  
22  
23 public void cekInformasi() {  
24     System.out.println("--- INFO REKENING ---");  
25     System.out.println("nO. Rekening : " + nomorRekening);  
26     System.out.println("Nama Pemilik : " + namaPemilik);  
27     System.out.println("Saldo Akhir : Rp" + saldo);  
28     System.out.println("-----");  
29 }  
30 }  
31 }
```

Baris 14 → `public void setorTunai(double nominal) {` digunakan untuk membuat method `setorTunai()` yang berfungsi melakukan penyetoran uang ke rekening.

Baris 15 → `if (nominal > 0) {` digunakan untuk memeriksa apakah nominal uang yang disetor lebih besar dari 0.

Baris 16 → `saldo += nominal;` digunakan untuk menambahkan nominal setoran ke saldo rekening.

Baris 17 → `System.out.println(...)` digunakan untuk menampilkan pesan bahwa setor tunai berhasil dan menampilkan saldo saat ini.

Baris 18 → `} else {` digunakan untuk menjalankan kondisi lain apabila nominal setoran tidak lebih dari 0.

Baris 19 → `System.out.println(...)` digunakan untuk menampilkan pesan bahwa setor tunai gagal karena nominal harus lebih dari 0.

Baris 20 → `}` digunakan untuk menutup percabangan if-else.

Baris 23 → `public void cekInformasi() {` digunakan untuk membuat method `cekInformasi()` yang berfungsi menampilkan informasi rekening.

Baris 24 → `System.out.println("--- INFO REKENING ---");` digunakan untuk menampilkan judul informasi rekening.

Baris 24 → `System.out.println("nO. Rekening : " + nomorRekening);` digunakan untuk menampilkan nomor rekening yang tersimpan pada variabel `nomorRekening`.

Baris 25 → `System.out.println("\nO. Rekening : " + nomorRekening);` digunakan untuk menampilkan nomor rekening yang tersimpan pada variabel `nomorRekening`.

Baris 26 → `System.out.println("Nama Pemilik : " + namaPemilik);` digunakan untuk menampilkan nama pemilik rekening.

Baris 27 → `System.out.println("Saldo Akhir : Rp" + saldo);` digunakan untuk menampilkan saldo terakhir yang dimiliki rekening.

Baris 28 → `System.out.println("-----");` digunakan untuk menampilkan garis pembatas pada informasi rekening.

Baris 29 → `}` digunakan untuk menandai akhir dari method `cekInformasi()`.

Baris 30 → `}` digunakan untuk menandai akhir dari class `Rekening`.

b) Class Main

```
1 package pekan1;
2
3 import java.util.Scanner;
4
5 public class Main {
6     public static void main(String[] args) {
7         Scanner input = new Scanner(System.in);
8         Rekening akunAktif = null; // Objek belum diinisialisasi (null)
9         boolean isRunning = true;
10
11         System.out.println("=== SISTEM PERBANKAN MINI ===");
12
13         while (isRunning) {
14             System.out.println("\nMenu Utama:");
15             System.out.println("1. Buka Rekening Baru");
16             System.out.println("2. Setor Tunai");
17             System.out.println("3. Tarik Tunai");
18             System.out.println("4. Cek Informasi Rekening");
19             System.out.println("0. Keluar");
20             System.out.println("Pilih menu: ");
21
22             int pilihan = input.nextInt();
23             input.nextLine(); // Membersihkan buffer enter
24
25             switch (pilihan) {
26                 case 1:
27                     System.out.print("Masukkan No Rekening: ");
28                     String no = input.nextLine();
29                     System.out.print("Masukkan Nama Pemilik: ");
30                     String nama = input.nextLine();
31                     System.out.print("Masukkan Saldo Awal: ");
32                     double saldo = input.nextDouble();
33
34                     // Instalasi Object/ Menjalankan Konstruktor
35                     akunAktif = new Rekening(no, nama, saldo);
36                     break;
```

```

38         case 2:
39             if (akunAktif == null) {
40                 System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
41             } else {
42                 System.out.print("Masukkan nominal setor: ");
43                 double setor = input.nextDouble();
44                 akunAktif.setorTunai(setor); // Memanggil Behavior / method
45             }
46             break;
47
48         case 3:
49             System.out.println("Fitur ini akan kerjakan sebagai Tugas Mandiri.");
50             break;
51
52         case 4:
53             if (akunAktif == null) {
54                 System.out.println("Error: Anda belum membuka rekening!");
55             } else {
56                 akunAktif.cekInformasi();
57             }
58             break;
59
60         case 0:
61             isRunning = false;
62             System.out.println("Sistem ditutup. Terima Kasih!");
63             break;
64
65         default:
66             System.out.println("Pilihan tidak valid!");
67     }
68     }
69     input.close();
70 }
71 }

```

Berikut penjelasan Langkah-langkah kerjanya:

1. Method main()

`public static void main(String[] args)` → Merupakan method utama yang pertama kali dijalankan ketika program Java dijalankan. Seluruh proses sistem perbankan mini dilakukan di dalam method ini.

2. Variabel input

`Scanner input = new Scanner(System.in);` → Membuat objek Scanner bernama input yang digunakan untuk menerima masukan dari pengguna melalui keyboard.

3. Variabel akunAktif

`Rekening akunAktif = null;` → Membuat variabel akunAktif bertipe Rekening untuk menyimpan objek rekening yang sedang digunakan. Nilai awalnya null karena pengguna belum membuat rekening.

4. Variabel isRunning

`boolean isRunning = true;` → Membuat variabel bertipe boolean untuk menentukan apakah program masih berjalan. Nilai true berarti program tetap berjalan.

5. Method while

`while (isRunning)` → Digunakan untuk mengulang tampilan menu selama nilai `isRunning` masih true.

6. Variabel pilihan

`int pilihan = input.nextInt();` → Menyimpan pilihan menu yang dimasukkan oleh pengguna dalam bentuk bilangan bulat.

7. Method switch

`switch (pilihan)` → Digunakan untuk menentukan tindakan yang dilakukan berdasarkan menu yang dipilih pengguna.

8. Case 1 – Membuka Rekening

Pada case 1, program meminta pengguna memasukkan nomor rekening, nama pemilik, dan saldo awal. Data tersebut kemudian digunakan untuk membuat objek Rekening.

`akunAktif = new Rekening(no, nama, saldo);` → Membuat objek rekening baru menggunakan constructor Rekening, kemudian menyimpannya ke dalam variabel `akunAktif`.

9. Variabel no

`String no = input.nextLine();` → Menyimpan nomor rekening yang dimasukkan oleh pengguna.

10. Variabel nama

`String nama = input.nextLine();` → Menyimpan nama pemilik rekening yang dimasukkan oleh pengguna.

11. Variabel saldo

`double saldo = input.nextDouble();` → Menyimpan saldo awal yang dimasukkan oleh pengguna.

12. Case 2 – Setor Tunai

Pada case 2, program melakukan pengecekan apakah pengguna sudah memiliki rekening.

`if (akunAktif == null)` → Memeriksa apakah objek rekening belum dibuat.

Jika `akunAktif` bernilai `null`, program menampilkan pesan bahwa pengguna belum memiliki rekening. Jika tidak `null`, pengguna dapat melakukan setor tunai.

13. Variabel setor

`double setor = input.nextDouble();` → Menyimpan nominal uang yang ingin disetorkan oleh pengguna.

14. Method `setorTunai()`

`akunAktif.setorTunai(setor);` → Memanggil method `setorTunai()` dari objek `akunAktif` untuk menambahkan nominal setoran ke saldo rekening.

15. Case 3 – Tarik Tunai

Pada case 3, program menampilkan informasi bahwa fitur tarik tunai belum dibuat dan akan dikerjakan sebagai tugas mandiri.

16. Case 4 – Cek Informasi Rekening

Pada case 4, program memeriksa apakah pengguna sudah memiliki rekening. Jika sudah, program menjalankan method `cekInformasi()`.

akunAktif.cekInformasi(); → Memanggil method cekInformasi() dari objek akunAktif untuk menampilkan nomor rekening, nama pemilik, dan saldo.

17. Case 0 – Keluar

Pada case 0, nilai isRunning diubah menjadi false.

isRunning = false; → Menghentikan perulangan while sehingga program berhenti.

18. Case default

default → Digunakan untuk menangani pilihan menu yang tidak tersedia, kemudian menampilkan pesan "Pilihan tidak valid!".

19. Method input.close()

input.close(); → Menutup objek Scanner setelah program selesai digunakan.

Alur Kerja Program

main()

→ Membuat Scanner

→ Membuat akunAktif dengan nilai null

→ Menampilkan menu

→ Pengguna memilih menu

→ 1: Membuat objek Rekening

→ 2: Memanggil setorTunai()

→ 3: Menampilkan fitur yang belum tersedia

→ 4: Memanggil cekInformasi()

→ 0: Menghentikan program

Jadi, class Main berfungsi sebagai pengatur jalannya program dan interaksi dengan pengguna, sedangkan class Rekening berfungsi sebagai objek yang menyimpan data dan perilaku rekening.

c) Output Program

```
<terminated> Main [Java Application] C:\Users\THINKPAD X13 YOGA\p2\pool\plugi
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
1
Masukkan No Rekening: 12345
Masukkan Nama Pemilik: Gina
Masukkan Saldo Awal: 200.000
Rekening atas nama Gina berhasil dibuat dengan saldo Rp200.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
2
Masukkan nominal setor: 100.000
Setor tunai Rp100.0 berhasil. Saldo saat ini: Rp300.0
```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
3
Fitur ini akan kerjakan sebagai Tugas Mandiri.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
4
--- INFO REKENING ---
n0. Rekening : 12345
Nama Pemilik : Gina
Saldo Akhir : Rp300.0
-----

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
0
Sistem ditutup. Terima Kasih!

```

B. Tugas/Latihan Praktikum

1. Implementasi Fitur Tarik Tunai

- Tambahkan method tarikTunai(double nominal) ke dalam class Rekening.
- Aturan : Saldo tidak boleh menjadi minus dan minimal nominal Tarik tunai adalah 10.000. Jika pengguna mencoba menarik nominal yang lebih besar dari saldo saat ini atau nominal kecil dari 10.000, program tidak boleh crash/error, melainkan menampilkan peringatan ke konsol: "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp[nominal]" atau "Transaksi Gagal : Minimal nominal penarikan 10.000".
- Terapkan method ini dengan menu nomor 3 pada class Main.

```

23 public void tarikTunai(double nominal) {
24     if (nominal < 10000) {
25         System.out.println("Transaksi Gagal: Minimal nominal penarikan 10.000.");
26     } else if (nominal > saldo) {
27         System.out.println("Transaksi Gagal: Saldo tidak mencukupi. "
28             + "Saldo Anda: Rp" + saldo);
29     } else {
30         saldo -= nominal;
31         System.out.println("Tarik tunai Rp" + nominal
32             + " berhasil. Saldo saat ini: Rp" + saldo);
33     }
34 }

```

Kode ini ditambahkan untuk menambahkan method tarikTunai() pada class Rekening yang berfungsi untuk melakukan transaksi penarikan uang. Method ini mengatur agar nominal penarikan minimal Rp10.000 dan tidak boleh melebihi saldo yang tersedia. Jika memenuhi ketentuan, saldo akan berkurang sesuai nominal yang ditarik dan program menampilkan informasi bahwa transaksi berhasil.

Ganti kode pada case 3 menjadi seperti di bawah ini:

```

49         case 3:
50             if (akunAktif == null) {
51                 System.out.println("Error: Silakan pilih atau buka rekening terlebih dahulu!");
52             } else {
53                 System.out.print("Masukkan nominal tarik tunai: ");
54                 double tarik = input.nextDouble();
55
56                 akunAktif.tarikTunai(tarik);
57             }
58             break;
59

```

Maka sekarang output program akan seperti ini

```

<terminated> Main [Java Application] C:\Users\THINKPAD X13 YOGA\.p2\pool\plugins\
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
1
Masukkan No Rekening: 1234
Masukkan Nama Pemilik: Gina
Masukkan Saldo Awal: 200.000
Rekening atas nama Gina berhasil dibuat dengan saldo Rp200.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
2
Masukkan nominal setor: 100.000
Setor tunai Rp100.0 berhasil. Saldo saat ini: Rp300.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
3
Masukkan nominal tarik tunai: 100.000
Transaksi Gagal: Minimal nominal penarikan 10.000.

```

```

<terminated> Main [Java Application] C:\Users\THINKPAD X13 YOGA\.p2\pool\plug
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
2
Masukkan nominal setor: 200000000
Setor tunai Rp2.0E8 berhasil. Saldo saat ini: Rp2.000003E8

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
3
Masukkan nominal tarik tunai: 20000
Tarik tunai Rp20000.0 berhasil. Saldo saat ini: Rp1.999803E8

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
4
--- INFO REKENING ---
n0. Rekening : 1234
Nama Pemilik : Gina
Saldo Akhir : Rp1.999803E8
-----

```

2. Tantangan Multi-Akun

Saat ini, aplikasi Main hanya dapat menyimpan 1 akun (Rekening `akunAktif = null`;) . Jika pengguna memilih menu "Buka Rekening Baru" untuk kedua kalinya, akun lama akan tertimpa.

- Ubah arsitektur penyimpanan di `Main.java` menggunakan struktur data Collection. (Gunakan `ArrayList`).
- Ubah logika menu "Buka Rekening": Setiap kali pengguna membuka rekening baru, objek tersebut ditambahkan ke dalam `ArrayList`.
- (Bonus): Tambahkan menu baru untuk "Ganti Akun", di mana sistem akan meminta nomor rekening, kemudian mencari objek yang cocok di dalam `ArrayList` untuk dijadikan `akunAktif`

Tambahkan case 5 digunakan untuk mengganti atau memilih akun rekening yang sedang aktif. Program akan meminta pengguna memasukkan nomor rekening, kemudian mencari nomor tersebut di dalam `ArrayList<Rekening>`. Jika nomor rekening ditemukan, objek rekening tersebut akan dijadikan `akunAktif`, sehingga transaksi seperti setor tunai, tarik tunai, dan cek informasi dapat dilakukan pada akun yang dipilih.

```
58         case 5:
59             System.out.print("Masukkan No Rekening yang ingin digunakan: ");
60             String nomorCari = input.nextLine();
61             akunAktif = null;
62             for (Rekening rekening : daftarRekening) {
63                 if (rekening.nomorRekening.equals(nomorCari)) {
64                     akunAktif = rekening;
65                     break;
66                 }
67             }
68             if (akunAktif != null) {
69                 System.out.println("Akun berhasil dipilih.");
70                 System.out.println("Pemilik: "
71                     + akunAktif.namaPemilik);
72             } else {
73                 System.out.println("Rekening tidak ditemukan.");
74             }
75             break;
```

Maka Output program seperti dibawah ini:

```

Main [Java Application] C:\Users\THINKPAD X13 YOGA\.p2\pool\plugins\org.eclipse.j
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
1
Masukkan No Rekening: 1357
Masukkan Nama Pemilik: gina
Masukkan Saldo Awal: 200
Rekening atas nama gina berhasil dibuat dengan saldo Rp200.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
2
Masukkan nominal setor: 3000
Setor tunai Rp3000.0 berhasil. Saldo saat ini: Rp3200.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
3
Masukkan nominal tarik tunai: 1000
Transaksi Gagal: Minimal nominal penarikan 10.000.

```

```

Main [Java Application] C:\Users\THINKPAD X13 YOGA\.p2\pool\plugins\

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
4
--- INFO REKENING ---
n0. Rekening : 1357
Nama Pemilik : gina
Saldo Akhir : Rp3200.0
-----

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
5
Masukkan No Rekening yang ingin digunakan: 2468
Rekening tidak ditemukan.

```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
5
Masukkan No Rekening yang ingin digunakan: 1357
Akun berhasil dipilih.
Pemilik: gina

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep class, object, constructor, dan method dapat diterapkan dalam pembuatan aplikasi Sistem Perbankan Mini. Program mampu membuat rekening, melakukan setor tunai, tarik tunai, serta menampilkan informasi rekening. Selain itu, penggunaan `ArrayList<Rekening>` memungkinkan program menyimpan lebih dari satu akun sehingga rekening yang telah dibuat tidak tertimpa. Penambahan fitur Ganti Akun juga memungkinkan pengguna memilih rekening berdasarkan nomor rekening yang tersimpan di dalam `ArrayList`. Melalui praktikum ini, dapat dipahami bahwa interaksi antarobjek dan manipulasi state sangat penting dalam penerapan pemrograman berorientasi objek.

DAFTAR PUSTAKA

Nurfiah. (2026). *Modul praktikum 1 pemrograman berorientasi objek–Java*.
Fakultas Teknologi Informasi, Universitas Andalas.